

Count Subarray Sum Equals K | Prefix Sum + HashMap

Source: TrainUrBrainX – Shivam Shakya, "DSA Brain Training Series"

Topic: Counting subarrays whose sum equals a target `k`, using Prefix Sum + HashMap.

1. The Problem

Given an array `arr` and a target `k`, count how many **contiguous subarrays** have a sum exactly equal to `k`.

From the whiteboard in the video:

```
arr = [1, 2, 3, 4]
k = 6
```

If you list every contiguous subarray and its sum:

```
[1]      = 1
[1,2]    = 3
[1,2,3]  = 6  ✓ matches k
[1,2,3,4] = 10
[2]      = 2
[2,3]    = 5
[2,3,4]  = 9
[3]      = 3
[3,4]    = 7
[4]      = 4
```

Only **[1,2,3]** sums to 6, so the answer is **1**.

That listing approach works, but it's $O(n^2)$ (or $O(n^3)$ if you re-sum every subarray from scratch). The video's whole point is: **how do we do this in $O(n)$?**

2. The Brute Force (and why it's slow)

The naive way: for every starting index `i`, walk forward and keep adding elements, checking after each addition if the running total equals `k`.

```
def subarraySum_bruteforce(arr, k):
    count = 0
    n = len(arr)
    for i in range(n):
        total = 0
        for j in range(i, n):
            total += arr[j]
            if total == k:
                count += 1
    return count
```

This is $O(n^2)$ — for an array of size 1000, that's up to ~1,000,000 operations. Fine for small inputs, painful for large ones. This is exactly the gap the video is filling with **Prefix Sum + HashMap**.

3. The Core Idea: Prefix Sum

A **prefix sum** at index `i` is just: *"the sum of everything from the start of the array up to and including index `i`."*

```
arr    = [1, 2, 3, 4]
prefix = [1, 3, 6, 10]
```

- `prefix[0] = 1`
- `prefix[1] = 1+2 = 3`
- `prefix[2] = 1+2+3 = 6`
- `prefix[3] = 1+2+3+4 = 10`

Why prefix sums matter: the sum of *any* subarray from index `i+1` to `j` can be computed instantly as:

$$\text{sum}(i+1 \dots j) = \text{prefix}[j] - \text{prefix}[i]$$

Example: $\text{sum of arr}[1..2] = 2+3 = 5$. Using prefix sums: $\text{prefix}[2] - \text{prefix}[0] = 6 - 1 = 5$ ✓.

This is the same "avoid recomputation" trick behind the **Pivot Index** notes you already have — there, $\text{right_sum} = \text{total_sum} - \text{left_sum} - \text{arr}[i]$ avoids re-summing the right side every time. Here, it's the same philosophy: *precompute cumulative sums once, then answer range-sum queries in $O(1)$ instead of re-adding elements every time.*

4. The Leap: From "Difference" to "HashMap Lookup"

We want to know: **does there exist some earlier prefix sum P_i such that $P_j - P_i = k$?**

Rearranging that equation:

$$P_i = P_j - k$$

So at every index j , instead of scanning backward through all previous prefix sums to check if one of them equals $P_j - k$, we ask a hashmap: *"have I seen the value $P_j - k$ before, and how many times?"*

This is exactly the **two-sum pattern** you already worked through with hashmaps — instead of checking every pair (i, j) , you flip the question: *"what value would I need to have already seen to complete a match right now?"* Then you just look it up in $O(1)$.

Here, the "complement" isn't $\text{target} - \text{arr}[j]$ (like in Two Sum) — it's $\text{running_sum} - k$.

5. Why We Also Need $\{0: 1\}$ at the Start

If a subarray starting from index 0 itself sums to k , there's no "earlier prefix" to subtract — the whole prefix up to j already equals k . To handle this cleanly, we say: *"a prefix sum of 0 , occurring before the array starts, has happened once."*

```
seen = {0: 1}    # sum-so-far : how many times we've seen it
```

This way the formula `running_sum - k` still finds a match even when the qualifying subarray starts at index 0.

6. Full Walkthrough with the Video's Example

```
arr = [1, 2, 3, 4], k = 6
```

Initialize: `running_sum = 0`, `count = 0`, `seen = {0: 1}`

i	arr[i]	running_sum	need = running_sum - k	in seen?	count	seen (updated)
0	1	1	1 - 6 = -5	No	0	{0:1, 1:1}
1	2	3	3 - 6 = -3	No	0	{0:1, 1:1, 3:1}
2	3	6	6 - 6 = 0	Yes (count=1)	1	{0:1, 1:1, 3:1, 6:1}
3	4	10	10 - 6 = 4	No	1	{...,10:1}

Final answer: `count = 1` — matching subarray `[1,2,3]`.  Same answer as the brute-force listing, but computed with a single left-to-right pass.

The key moment is at `i = 2`: `running_sum` becomes `6`, and `need = 0` is found in `seen` because we pre-seeded `{0:1}`. That match represents the subarray `[1,2,3]` — everything from "the start" up to index 2.

7. The Code

```

from collections import defaultdict
from typing import List

class Solution:
    def subarraySum(self, arr: List[int], k: int) -> int:
        seen = defaultdict(int)
        seen[0] = 1          # empty prefix, sum = 0

        running_sum = 0
        count = 0

        for num in arr:
            running_sum += num
            need = running_sum - k
            count += seen[need]    # how many earlier prefixes match
            seen[running_sum] += 1 # record current prefix sum

        return count

```

Notice the structure is nearly identical to your `sumPrefixScores` solution in `Sum_of_Prefix_Score_of_string.py`:

- Both build a `defaultdict(int)` to track "how many times have I seen this key."
- Both make **two logical passes** over data conceptually — one to build up cumulative state (`hash_map[curr] += 1` there, `seen[running_sum] += 1` here), one to query it.
- The difference: your string problem counts **exact prefix matches**; this problem counts **difference-based matches** (`running_sum - k`), which is what makes it a "Two Sum on prefix sums" pattern rather than a direct lookup.

8. Order of Operations Matters

Just like the Pivot Index notes warned about updating `left_sum` *after* checking the condition, here the order is:

```
running_sum += num      # 1. update running sum first
need = running_sum - k   # 2. compute what we need
count += seen[need]      # 3. check/count BEFORE inserting current sum
seen[running_sum] += 1   # 4. THEN record current sum
```

If you recorded `seen[running_sum] += 1` *before* checking, you could accidentally count a "subarray" of length 0 (an element matching itself against itself), which isn't valid — a subarray must contain at least one element. Doing the lookup before the insert keeps each `running_sum` entry representing only sums seen **strictly before** the current index.

9. Complexity

Approach	Time	Space
Brute force (nested loops)	$O(n^2)$	$O(1)$
Prefix Sum + HashMap	$O(n)$	$O(n)$

We trade a bit of memory (storing prefix sum frequencies) for a massive time improvement — one pass instead of nested loops.

10. The Pattern to Remember

Whenever a problem says:

"Count subarrays whose sum equals X" / "find subarrays with sum = k" / "sum of subarray between two indices equals target"

Think:

```
running_sum accumulates as you scan left to right
need = running_sum - k
answer += how many times "need" has occurred before
then record running_sum itself
```

This is the **same family** as:

- **Two Sum** (hashmap of complements instead of nested loops)
- **Pivot Index** (prefix sums to avoid recomputation)
- **Sum of Prefix Scores** (hashmap tracking frequency of prefixes)

All three are variations of: *"precompute a running/cumulative value once, then use a hashmap to answer 'have I seen what I need' in $O(1)$ instead of re-scanning."*

11. Quick Recap (one-line memory trick)

"Sum of subarray ($i+1 \rightarrow j$) = `running_sum(j)` – `running_sum(i)`. So instead of finding pairs, just ask the hashmap: have I already seen `running_sum – k`?"